

EPIC LEARN EDUCATION · GO PROGRAMMING MASTER COURSE

Week 1, Day 1

Software engineering, why Go, your toolchain,
and the first program you will put on GitHub.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

WHO IS
TEACHING

Umesh Indrajith

Background

BSc Engineering (Hons) in Electronic and Telecommunication Engineering, University of Moratuwa.

Practice

Senior Software Engineer. You will learn how Go is used to ship backend systems, not only how the language looks.

This room

Ask questions as we go. If the install fails, say so immediately — do not sit broken until the lab.

Contact

Epic Learn Education
epiclearneducation.com
074 1200 659

Software engineering, using Go

- Not “memorise syntax for 16 weeks, then a project at the end”.
- You will learn to **design, build, test, review, and explain** working software.
- Go is the language. Git, HTTP, databases, and tests are the job.
- By week 16 you have a portfolio API: the Student Management System.

16 WEEKS The shape of the course

Weeks 1-3

Language enough to be dangerous: types, control flow, slices, maps, first functions, Git every week.

Weeks 4-16

One product: SMS. Health endpoint, then students, then Postgres, then login. Ten to twelve weeks of hands-on.

Why REST so early?

So you have something to demo from week 5. We deepen functions and structs *inside that project*.

Honest promise

Junior backend foundation. Not Kubernetes. Not “master of cloud”.

TODAY

If Day 1 works, you can

- Explain what software engineering is, in two sentences.
- Say why we chose Go for a backend course.
- Run Go 1.25, VS Code, and gofmt on Ubuntu (or WSL).
- Write, run, and build Hello, Epic Learn.
- Put that program on GitHub.

How we will use the time

- **0:00–0:50** Ideas: software engineering and why Go
- **0:50–1:40** Live: install Go and set up VS Code
- **1:40–2:20** Live: first program, `go run`, `go build`, `gofmt`
- **2:20–3:00** Git + GitHub
- **3:00–end** Lab: you do it, I walk the room

Building software that other people can trust

Programming is writing instructions.
Software engineering is delivering a system
that can be changed, tested, and run by a
team — including future you.

- It has users, constraints, and a lifespan longer than the demo.
- It has quality: correctness, readability, security of secrets, tests.
- It has a process: we do not only “code until it works on my machine”.

THE LOOP WE
WILL USE ALL 16
WEEKS

**Requirements → design
→ implement → test →
review**

Requirements

Who is it for? What must it do? What is out of scope?

Design

Folders, API shape, data. A little design beats a mess.

Implement

Go code. Small steps. Commit often.

Test & review

go test, then another human reads it. That is the job.

How we work here

- I type, you type. Watching is not the skill.
- Homework is a **GitHub pull request**, not a zip on WhatsApp.
- Unformatted code comes back. gofmt is not optional.
- AI tools are allowed. If you cannot explain the line, it is not yours.

WHY GO **A language designed for building services**

Created at Google (2007–2009) for large systems: simple language, excellent toolchain, great concurrency, one static binary.

- Used for APIs, CLIs, cloud tooling, Kubernetes-related systems.
- Fast to compile. Easy to read. Hard to hide complexity behind magic.
- We chose it because a beginner can reach a real HTTP API in this course.

What we are buying

Simplicity

Small language. Fewer ways to write the same thing. Good for teams.

Toolchain

go run, go build, gofmt, go test, go mod — one installer.

Concurrency

Goroutines. Weeks 10–12.
Mention now, practise later.

One binary

Compile and copy. No virtualenv ritual to run the server.

Standard library

HTTP and JSON live in stdlib. We add Gin once you have seen net/http.

Garbage collection

You will still learn pointers. You will not manage malloc by hand.

Student Management System (SMS)

- Staff register and log in.
- Staff create, list, update, delete students.
- Go API + PostgreSQL. A React UI is provided — you are graded on Go.
- **Not today.** Weeks 1–3 are the tools. Project kickoff is week 4.

Everyone uses the same versions

- **Go 1.25.x** from go.dev — not whatever `apt install golang-go` gives you.
- **Ubuntu** in the lab. Windows: WSL2 + Ubuntu. macOS is fine.
- **VS Code** + official Go extension. GoLand is optional if you already have it.
- Git + a GitHub account. Postman comes when we do HTTP.

LIVE
DEMO

INSTALL GO 1.25 ON
UBUNTU

Do this with me

```
wget https://go.dev/dl/go1.25.0.linux-amd64.tar
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.25.0.linux-amd6
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~
source ~/.bashrc
go version
```

DID IT
WORK?

go version and go env

```
go version
go env GOROOT GOPATH GOMOD
```

- GOROOT — where the Go install lives (usually /usr/local/go).
- GOPATH — old workspace idea. We **do not** put class projects there.
- **Modules** — a folder with go.mod is the project. That is week 4. Today: one file is enough.

VS Code, set up for Go

- Install VS Code if needed. Open it.
- Extensions: search **Go** by the Go Team at Google. Install it.
- Open a folder you will use for this course, e.g. ~/epic-go.
- Command Palette: **Go: Install/Update Tools** — install all (gofmt, gopls, dlv).

THE SETTING
YOU MUST
TURN ON

Format on save = gofmt

```
{  
  "editor.formatOnSave": true,  
  "[go]": {  
    "editor.defaultFormatter": "golangci.gofmt",  
    "editor.formatOnSave": true  
  }  
}
```

Settings JSON (Ctrl+Shift+P → "Preferences: Open User Settings (JSON)"). From today, unformatted Go is rejected.

Anatomy of a Go file

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, Epic!")
}
```

■ package main + func main → an executable program.

■ import "fmt" — standard library formatted I/O.

■ Save as hello.go.

Exported vs unexported

- A name that starts with a **capital letter** is exported (visible from other packages).
- `fmt.Println` works because `Println` is exported.
- `fmt.println` will not compile.
- This is not Java's public keyword. The capital letter *is* the rule.

```
fmt.Println("ok")  
  
// fmt.println("no")  
// undefined: fmt.println
```

Three commands

```
gofmt -w hello.go      # rewrite
go run hello.go         # compile
go build -o hello hello.go # produce
./hello                 # run the
```

- go run — while learning.

- go build — what you ship (a file you can copy).

- go test exists. We use it on a real function in week 2.

Why Git on day 1

If it is not on GitHub, it is a story you told. The course artefact is the history of your work: commits, then pull requests from week 4.

Init, ignore, commit

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

git init
echo -e "hello\n*.exe" > .gitignore # we will improve
git add hello.go .gitignore
git status
git commit -m "Add Hello Epic Learn"
```

A remote is a backup and a classroom

```
git branch -M main  
git remote add origin https://  
git push -u origin main
```

- Create an empty repo on github.com (no README if you already committed locally).
- Add me as collaborator if that is how this batch submits.
- `git clone` is how you copy a repo onto a new machine — we will use it for SMS in week 4.

Done when all of this is true

- `go version` shows 1.25.x
- VS Code formats a Go file on save
- `hello.go` prints Hello, Epic Learn via `go run` and via `./hello`
- GitHub repo contains `hello.go`, not the compiled binary
- Show me the GitHub URL before you leave

AFTER
CLASS

Homework (due before next session)

- Half page: **Why we will use Go for the Student Management System.** Your words. Submit as why-go.md in the same repo.
- Screenshot of go version in the terminal. Add it to the repo or the classroom chat as instructed.
- Optional: change the message to include your name, commit again, push.

Week 1 wrap / Week 2 start

- If anyone's install is still broken, we fix it first.
- Then: variables, types, zero values, operators, strings.
- Bring a working go version and your GitHub URL.

Questions.

Then open the terminal. We do not
leave until go version works.

Umesh Indrajith · Go Programming
Master Course · Week 1, Day 1